总体目标： AI 生成候选证明， Lean 独立验收

实验目标：用 Lean 4 + Mathlib 把「对任意正整数 n，存在素数 p > n」形式化，并让机器独立验证

主命题（Euclid 定理）：对任意正整数 n，存在素数 p > n

示例： n = 3 时取 p = 5； n = 10 时取 p = 11； n = 100 时取 p = 101

思路： 取 n! + 1 的素因子 p；若 p ≤ n，则 p ∣ n! 且 p ∣ n! + 1，故 p ∣ 1，矛盾

意义： 这是构造性证明，没有「打表」捷径，正好用来检验 AI 的多步推理与自我修正能力

分工： AI 负责提出候选证明， Lean 负责独立裁决 —— 谁都不能自己说「对了」

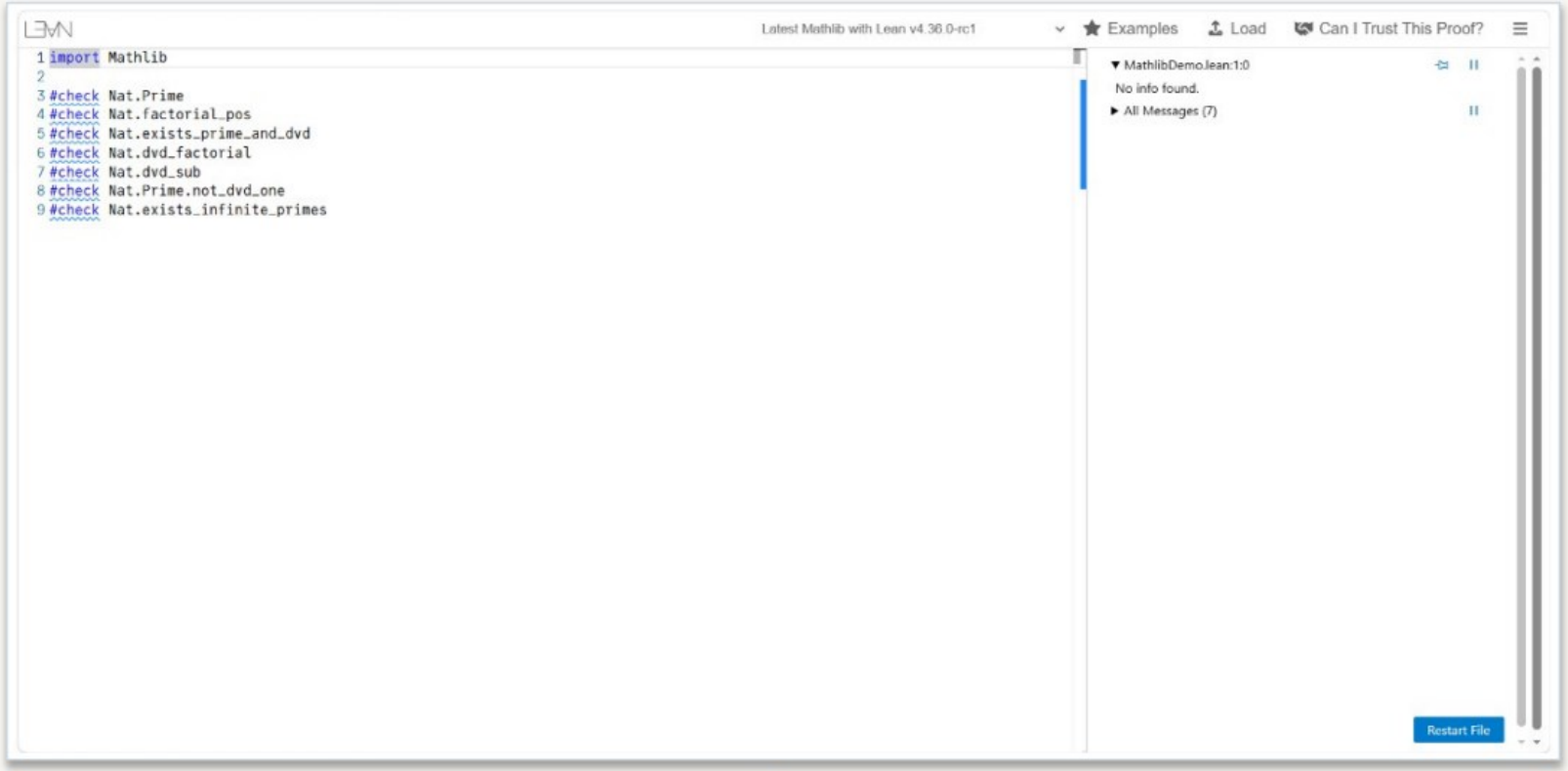
通过： 0 error + 0 warning，且公理依赖只含三条标准公理

执行步骤：四步依次对应第 2 页、第 3–4 页、第 5 页、第 6 页

S1 验证 Lean 环境 → S2 生成候选代码 → S3 查看验证结果 → S4 修订并重检

1 第一步：验证 Lean 在线环境

打开 live.lean-lang.org 并选带 Mathlib 的项目；先用 #check 把要用的引理逐个核对一遍，再写证明



在线 Lean

live.lean-lang.org

- 选 Latest Mathlib with Lean v4.36.0-rc1
- 左侧写代码，右侧消息区给结论
- 免安装，浏览器里直接跑

本地等价环境

elan + Lean 4 + Mathlib （lake exe cache get）；本地受阻时随时退回在线版

版本已验证

Latest Mathlib + Lean v4.36.0-rc1， import Mathlib 可用

7 条引理全部命中

先 #check 再写，避免 application type mismatch 连环报错

粘贴即检查

左侧写代码、右侧看消息，不用手敲命令

2 第二步：输入提示词，生成候选代码

一条提示词同时交代题目、声明、禁止项与自检项，并要求只输出一个 Lean 代码块

输入

- 命题：任意正整数 n，要求 p > n
- 声明： theorem 一行一字不差
- 禁止： sorry 、 admit 、新增 axiom
- 自检：末尾 #print axioms

AI 生成

输出

- 5 行以内的数学解释
- 一个可直接编译的代码块

请证明：对任意正整数 n，都存在素数 p，使得 p > n。

要求：
1. 先给 5 行以内的数学解释；
2. 再给完整、可直接编译的 Lean 4 代码；
3. 以 import Mathlib 开头，定理声明必须一字不差：
theorem exists_prime_gt (n : ℕ) (hn : 0 < n) : ∃ p : ℕ, n < p ∧
p.Prime := by
4. 不得使用 sorry、admit，不得新增 axiom；
5. 末尾加一行 #print axioms exists_prime_gt；
6. 只输出一个 lean 代码块。

实际使用的提示词（截图存档，未改写）

对应关系：提示词里的六条约束，正是后面验收清单的来源
声明一字不差 → 命题一致 | 禁 sorry → 无 sorryAx | #print axioms → 公理可审

2 第二步： AI 给出的候选代码

声明与板书一字不差；用 $n! + 1$ 的素因子构造 p，再反设 $p \le n$ 导出矛盾；末尾自带公理自检

```
import Mathlib

theorem exists_prime_gt (n : ℕ) (hn : 0 < n) :
    ∃ p : ℕ, n < p ∧ p.Prime := by
  have hm : Nat.factorial n + 1 ≠ 1 := by
    have hpos : 0 < Nat.factorial n :=
      Nat.factorial_pos n
    omega
  obtain ⟨p, hp_prime, hp_dvd⟩ :=
    Nat.exists_prime_and_dvd
      (n := Nat.factorial n + 1) hm
  refine ⟨p, ?_, hp_prime⟩
  by_contra h
  push_neg at h
  have hp_dvd_fact : p ∣ Nat.factorial n :=
    Nat.dvd_factorial hp_prime.pos h
  have hp_dvd_one : p ∣ 1 :=
    (Nat.dvd_add_iff_right hp_dvd_fact).mpr hp_dvd
  exact hp_prime.not_dvd_one hp_dvd_one

#print axioms exists_prime_gt
```

① 数学注释写在最前面

顶部 5 行把 Euclid 原证的推理链交代清楚

② 声明与板书一致

$n : \mathbb{N}$，结论 $n < p \wedge p.\text{Prime}$，未被简化

③ 构造走 n! + 1

obtain 取素因子 p， by_contra 反设 $p \le n$

④ 末尾自带公理自检

#print axioms 写在文件末尾，验收直接读

留意：声明里的 hn 是冗余假设

n = 0 时取 p = 2 也成立，所以 hn 并未被引用 —— 下一屏 Lean 就会就此报警告。

3 第三步：查看验证结果（第一次编译）

0 error，但消息区出现 2 条 linter 警告 —— 这还不能算通过，必须退回修订



Lean 消息区（原文）

```
MathlibDemo.lean:3:33
Variable name `hn` is not
explicitly referenced.

MathlibDemo.lean:11:2
`push_neg` has been deprecated.
Prefer using `push Not` instead.
```

0 error / 2 warnings —— 不算通过

判定：本轮不通过

2 条 linter 警告没有清零。「没有报错」只是编译层面的结论，不等于验收通过。

下一步：带着消息原文退回修订

把代码与消息原文一起回传 AI，要求只改被指出的两处，声明一个字符都不动。

4 第四步：按反馈修订，重新检查

两处改动都不触碰定理声明；改完立刻重跑，直到 0 error / 0 warning

① 抑制「hn 未使用」警告

```
set_option linter.unusedVariables false in
```

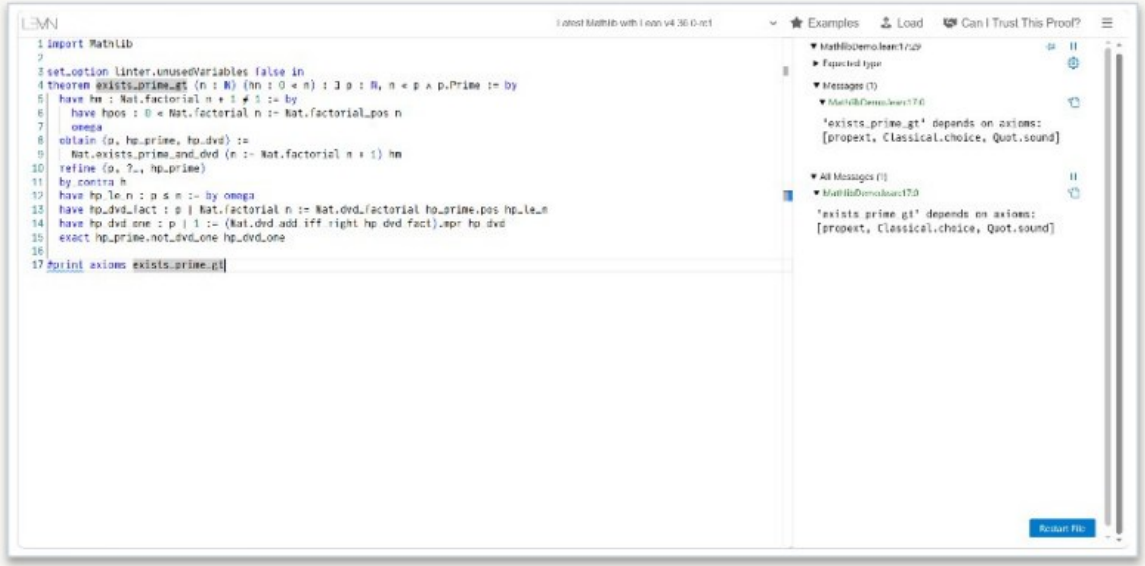
不动声明 —— 这是「命题不可变」原则的可执行版本

② push_neg 换成 omega

```
have hp_le_n : p ≤ n := by omega
have hp_dvd_fact : p ∣ Nat.factorial n :=
  Nat.dvd_factorial hp_prime.pos hp_le_n
```

直接拿到 p ≤ n，不再依赖已废弃的 push_neg

重新检查后： 0 error / 0 warning



公理依赖（原文）

```
[propext,
 Classical.choice,
 Quot.sound]
```

第 2 轮通过

只含三条标准公理，没有 sorryAx 。

结论：把人工循环固定下来，才是可复现的证明

本次并非「一次通过」——第 2 轮才通过；但代码与消息原文都在，结论才站得住

验收清单（写进报告）

① **命题与原题一致**

声明一字未改： n : ℕ ， 结论 n < p ∧ p.Prime ； 改的只有 tactic

② **0 error / 0 warning**

第一次的 2 条 linter 警告已清零，编译干净

③ **无 sorryAx 、无自定义公理**

公理只剩 [propext, Classical.choice, Quot.sound]

可复现要保存的四样东西

- **首次代码**

  AI 的第一版，含被指出的问题

- **报错原文**

  Lean 消息区的文本或截屏

- **每轮改动**

  改了什么、为什么改

- **最终结果**

  代码 + #print axioms 输出

认识： AI 生成候选、 Lean 独立裁决 —— 把中间的「人肉复制粘贴」固定成流程，证明才可复现、可审计